

CollectionView

La classe `CollectionView` est conçue pour afficher des listes dans des mises en page complexes et fournit une alternative plus flexible et plus performante à `ListView`.

Différences avec `ListView` :

- **CollectionView** dispose d'un modèle de mise en page flexible, vous permettant d'afficher les données verticalement ou horizontalement, sous forme de liste ou de tableau.
- **CollectionView** prend en charge la sélection d'éléments simples et multiples.
- **CollectionView** n'utilise pas le concept de cellules comme `ListView`. Il utilise plutôt un modèle de données pour déterminer l'apparence de chaque élément.
- **CollectionView** utilise automatiquement la virtualisation fournie par les éléments sous-jacents natifs du système d'exploitation sous-jacent.
- **CollectionView** possède une API plus restreinte. De nombreuses propriétés et événements de `ListView` sont absents de `CollectionView`.
- **CollectionView** ne prend pas en charge les séparateurs d'éléments.
- **CollectionView** lèvera une exception si sa propriété `ItemsSource` est mise à jour en dehors du thread de l'interface utilisateur.

Définition de la source de données

Pour définir une source de données, utilisez la propriété **ItemsSource** , qui prend une collection comme valeur `IEnumerable`. Par exemple, définissons la page suivante en C# :

```
1 class StartPage : ContentPage
2 {
3     public StartPage()
4     {
5         CollectionView collectionView = new CollectionView {
6             Margin=5 };
7         collectionView.ItemsSource = new string[] { "Tom", "Sam",
8             "Bob", "Alice", "Kate" };
9         Content = collectionView;
10    }
```



Configuration d'un modèle de données

La propriété **ItemsTemplate**, qui représente un objet **DataTemplate** et prend comme valeur l'objet **DataTemplate** du même nom, est responsable de la définition du modèle d'affichage des données. Un exemple simple :

```
1 class StartPage : ContentPage
2 {
3     public StartPage()
4     {
5         CollectionView collectionView = new CollectionView { Margin=5 };
6         // nous déterminons la source des données
7         collectionView.ItemsSource = new string[] { "Tom", "Sam", "Bob", "Alice", "Kate" };
8         // nous définissons le modèle de données
9         collectionView.ItemTemplate = new DataTemplate(() =>
10     {
11         var personLbl = new Label { FontSize = 16, TextColor = Color.FromArgb("#1565C0") };
12         personLbl.SetBinding(Label.TextProperty, new Binding("BindingContext", source:
13 RelativeBindingSource.Self));
14         return personLbl;
15     });
```

```

16     Content = collectionView;
17 }
    }

```

Le constructeur `DataTemplate` accepte un délégué qui renvoie un objet modèle d'élément de liste. Dans ce cas, l'étiquette `personLbl` sert de modèle d'élément, dont la couleur du texte et la hauteur de police sont définies. Le contexte de liaison par défaut de l'objet modèle (la propriété `BindingContext`) étant l'élément de liste affiché, le texte de l'étiquette est affiché par liaison à `BindingContext` la propriété de cette même étiquette (`new Binding("BindingContext", source: RelativeBindingSource.Self)`)

Exemple similaire en xaml :

```

1 <?xml version="1.0" encoding="utf-8" ?>
2 <ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
3     xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
4     x:Class="HelloApp.MainPage" >
5     <CollectionView>
6         <CollectionView.ItemsSource>
7             <x:Array Type="{x:Type x:String}">
8                 <x:String>Tom</x:String>
9                 <x:String>Sam</x:String>
10                <x:String>Bob</x:String>
11                <x:String>Alice</x:String>
12                <x:String>Kate</x:String>
13            </x:Array>
14        </CollectionView.ItemsSource>
15        <CollectionView.ItemTemplate>
16            <DataTemplate>
17                <Label Text="{Binding}" FontSize="16" TextColor="#1565C0" />
18            </DataTemplate>
19        </CollectionView.ItemTemplate>
20    </CollectionView>

```

```
21</ContentPage>
```

Définir un modèle pour les données complexes

Les objets de liste représentent généralement des structures de données plus complexes. Par exemple, supposons une classe **Person** représentant un utilisateur :

```
1 public class Person
2 {
3     public string Name { get; set; } = "";
4     public int Age { get; set; }
5     public string Company { get; set; } = "";
6 }
```

Dans le code C#, nous définissons une page qui affiche une liste d'utilisateurs dans une `CollectionView` :

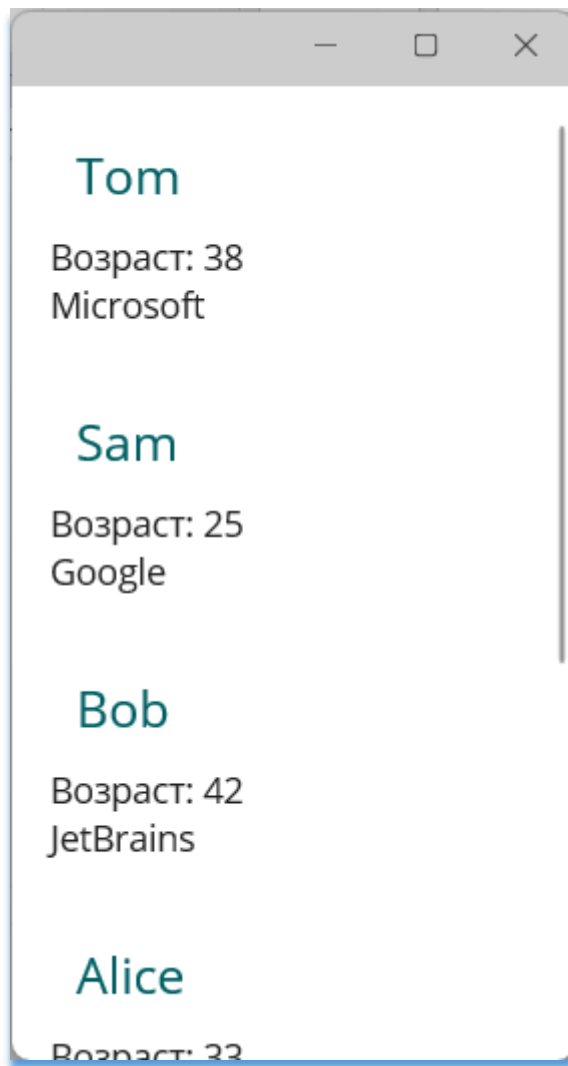
```
1 namespace HelloApp;
2
3 class StartPage : ContentPage
4 {
5     public StartPage()
6     {
7         CollectionView collectionView = new CollectionView();
8         // определяем источник данных
9         collectionView.ItemsSource = new List<Person>
10     {
11         new Person { Name="Tom", Age=38, Company="Microsoft" },
12         new Person { Name="Sam", Age=25, Company="Google" },
13         new Person { Name="Bob", Age=42, Company="JetBrains" },
14         new Person { Name="Alice", Age=33, Company="Microsoft" },
15         new Person { Name="Kate", Age=29, Company="Google" },
16         new Person { Name="Amelia", Age=35, Company="JetBrains" },
17     };
18     // определяем шаблон данных
```

```

19  collectionView.ItemTemplate = new DataTemplate(() =>
20  {
21      var nameLabel = new Label { FontSize = 20, TextColor = Color.FromArgb("#006064"), Margin = 10 };
22      nameLabel.SetBinding(Label.TextProperty, "Name");
23
24      var ageLabel = new Label();
25      ageLabel.SetBinding(Label.TextProperty, new Binding { Path = "Age", StringFormat = "Возраст: {0}" });
26
27      var companyLabel = new Label();
28      companyLabel.SetBinding(Label.TextProperty, "Company");
29
30      return new StackLayout
31      {
32          Children = { nameLabel, ageLabel, companyLabel },
33          Margin = new Thickness(15, 10)
34      };
35  });
36  Content = collectionView ;
37  }
38}

```

Dans ce cas, l'objet StackLayout sert de modèle de données . Chaque élément se voit attribuer son propre StackLayout, avec trois étiquettes, chacune liée à une propriété spécifique de l'objet Person. Sous Windows, cela ressemblerait à ceci :



Exemple similaire en xaml :

```
1 <?xml version="1.0" encoding="utf-8" ?>
2 <ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
3     xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
4     x:Class="HelloApp.MainPage"
5     xmlns:local="clr-namespace:HelloApp">
6     <CollectionView>
7         <CollectionView.ItemsSource>
8             <x:Array Type="{x:Type local:Person}">
9                 <local:Person Name="Tom" Age="38" Company="Microsoft"/>
10                <local:Person Name="Sam" Age="25" Company="Google"/>
11                <local:Person Name="Bob" Age="42" Company="JetBrains"/>
```

```

12      <local:Person Name="Alice" Age="33" Company="Microsoft"/>
13      <local:Person Name="Kate" Age="29" Company="Google"/>
14      <local:Person Name="Amelia" Age="35" Company="JetBrains" />
15      </x:Array>
16  </CollectionView.ItemsSource>
17  <CollectionView.ItemTemplate>
18      <DataTemplate>
19          <StackLayout Margin="8">
20              <Label Text="{Binding Name}" FontSize = "20" TextColor = "#006064" Margin="10" />
21              <Label Text="{Binding Age, StringFormat='Возраст: {0}'}" />
22              <Label Text="{Binding Company}" />
23          </StackLayout>
24      </DataTemplate>
25  </CollectionView.ItemTemplate>
26 </CollectionView>
27</ContentPage>

```

En-tête et pied de page de CollectionView

Grâce **aux propriétés « En-tête »** et **« Pied de page »** de CollectionView, vous pouvez définir respectivement l'en-tête et le pied de page. Nous pouvons définir les valeurs comme des chaînes standard :

Par exemple, dans xaml :

```

1 <?xml version="1.0" encoding="utf-8" ?>
2 <ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
3     xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
4     x:Class="HelloApp.MainPage"
5     xmlns:local="clr-namespace:HelloApp">
6     <CollectionView Header=" Liste des utilisateurs " Footer=" Les données sont à jour en date de
7     ..... ">
8     <CollectionView.ItemsSource>
9         <x:Array Type="{x:Type local:Person}">
10             <local:Person Name="Tom" Company="Microsoft" />
11             <local:Person Name="Bob" Company="Google" />

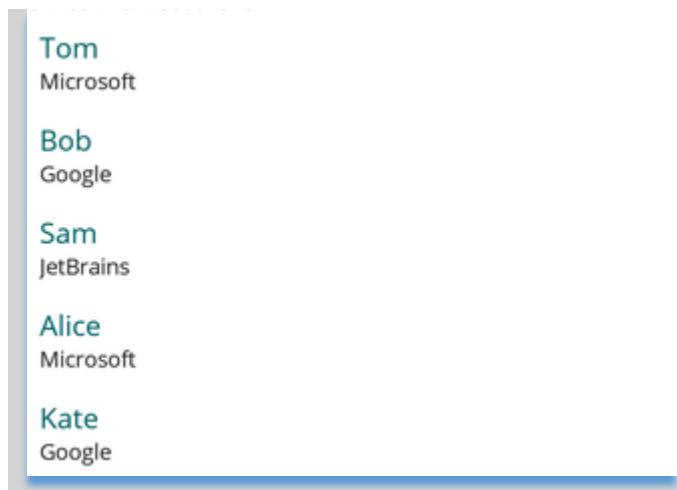
```

```

11      <local:Person Name="Sam" Company="JetBrains" />
12      <local:Person Name="Alice" Company="Microsoft" />
13      <local:Person Name="Kate" Company="Google" />
14  </x:Array>
15 </CollectionView.ItemsSource>
16 <CollectionView.ItemTemplate>
17     <DataTemplate>
18         <StackLayout Padding="8">
19             <Label Text="{Binding Name}" FontSize="18" TextColor="#006064" />
20             <Label Text="{Binding Company}" />
21         </StackLayout>
22     </DataTemplate>
23 </CollectionView.ItemTemplate>
24 </CollectionView>
25</ContentPage>

```

Mais dans ce cas, nous verrons de petites inscriptions au début et à la fin de la liste :



Installation en code C# :

```

1CollectionView collectionView = new CollectionView();
2collectionView.Header = "Список пользователей";
3collectionView.Footer = "Date : ..... ";

```

CollectionView vous permet également de spécifier un modèle pour les afficher :

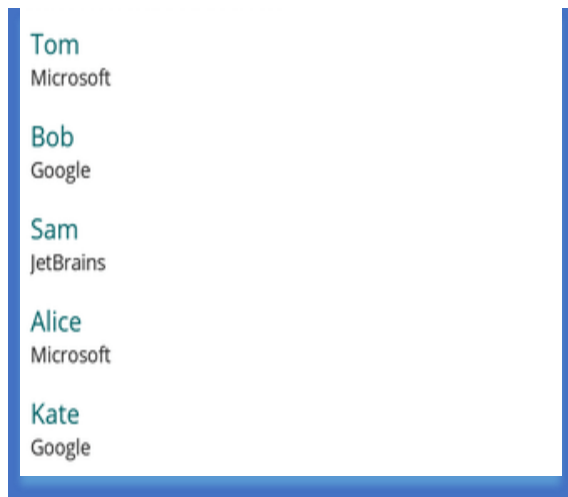
```

1 <?xml version="1.0" encoding="utf-8" ?>

```



```
2 <ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
3     xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
4     x:Class="HelloApp.MainPage"
5     xmlns:local="clr-namespace:HelloApp">
6     <CollectionView>
7         <CollectionView.ItemsSource>
8             <x:Array Type="{x:Type local:Person}">
9                 <local:Person Name="Tom" Company="Microsoft" />
10                <local:Person Name="Bob" Company="Google" />
11                <local:Person Name="Sam" Company="JetBrains" />
12                <local:Person Name="Alice" Company="Microsoft" />
13                <local:Person Name="Kate" Company="Google" />
14            </x:Array>
15        </CollectionView.ItemsSource>
16        <CollectionView.Header>
17            <Label Text= " Liste des utilisateurs " FontSize="18" FontAttributes="Bold" />
18        </CollectionView.Header>
19        <CollectionView.Footer>
20            <Label Text= " Janvier 2023r." FontSize="14" FontAttributes="Italic" />
21        </CollectionView.Footer>
22        <CollectionView.ItemTemplate>
23            <DataTemplate>
24                <StackLayout Padding="8">
25                    <Label Text="{Binding Name}" FontSize="18" TextColor="#006064" />
26                    <Label Text="{Binding Company}" />
27                </StackLayout>
28            </DataTemplate>
29        </CollectionView.ItemTemplate>
30    </CollectionView>
31</ContentPage>
```



Travail supplémentaires :

1. Afficher pour chaque personne une photo
2. Modifier la « collectionview » pour afficher la liste horizontalement
3. Modifier le fonctionnement comme suit :
 - a. Ajouter SQLite à votre application
 - b. Les données de la « collectionview » doivent avoir comme source une base de données locale SQLite.

Lien sur l'intégration de SQL lite avec MAUI : [Dot.Blog | SQLite avec MAUI : Stockage de Données Locales](#)